

**SIMATS ENGINEERING (SAVEETHA SCHOOL
OF ENGINEERING) Department of Computer
Science and Engineering**

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	JAYAKRISHNA.V	Reg. No.:	192421259
Section / Batch:	I	Date:	8.8.26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I __ (JAYAKRISHNA.V / 192421259) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JAYAKRISHNA

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question

Online Book Search System (Binary Search): An online bookstore maintains a sorted catalog of books and needs fast search functionality.

Simple Explanation

This question is asking you to **speed up matrix multiplication used in image compression**.

1. What is the problem?

Images can be represented as **matrices of numbers (pixels)**.

For example:

Image Matrix A Transformation Matrix B

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Matrix multiplication is required during image transformation/compression.

For **large images**, the matrices become very large, so normal matrix multiplication takes a lot of time.

2. Computational Challenge

Traditional matrix multiplication uses **three nested loops**:

```
for i
  for j
    for k
      C[i][j] += A[i][k] × B[k][j]
```

For an $n \times n$ matrix:

Time Complexity = $O(n^3)$

So, when n becomes very large, the computation becomes slow.

3. Divide and Conquer Approach

Divide the large matrices into smaller submatrices.

For example:

Matrix A

A11		A12
A21	A22	

Similarly divide Matrix B.

Then:

Divide the matrices.

Multiply the smaller matrices

Combine the results

This is called **Divide and Conquer**.

4. Strassen's Matrix Multiplication

Strassen's algorithm improves matrix multiplication by reducing the number of recursive multiplications.

Traditional method requires:

8 multiplications

for two divided matrices.

Strassen's method requires only:

7 multiplications

This reduction makes it faster for very large matrices.

Time Complexity

Method	Time Complexity
Traditional Matrix Multiplication	$O(n^3)$
Divide & Conquer	$O(n^3)$
Strassen's Algorithm	$O(n^{2.807})$

So, Strassen's algorithm is theoretically faster for large matrices.

5. Proposed Solution

For the image compression engine:

Input Image



Convert Image → Matrix



Divide Large Matrix



Strassen Matrix Multiplication



Combine Results



Compressed Image

Why is it efficient?

Reduces the number of matrix multiplications.

Faster for **large matrices**.

Uses a Divide and Conquer strategy.

Reduces computational time compared with traditional $O(n^3)$ multiplication.

Final Justification

Strassen's algorithm is a suitable optimization for an image compression system because image transformations can involve very large matrices. Its time complexity of approximately $O(n^{2.807})$ is better than the traditional $O(n^3)$, providing improved performance for large-scale matrix operations.

Output :

CO3 AT2-CASE STUDY

[← Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

Q1

Online Book Search System
(Binary Search) An online bookstore maintains a sorted catalog of books and needs fast search functionality.
100 marks

1/1 answered

Q1 Online Book Search System (Binary Search) An online bookstore maintains a sorted catalog of books and needs fast search functionality 100 marks

Analyze how Binary Search improves search efficiency. Identify requirements such as sorted indexing and quick access. Apply divide and conquer principles to explain the process. Analyze time complexity and justify why Binary Search is appropriate.

Your Code

```
1 def binary_search(arr, target):
2     low, high = 0, len(arr) - 1
3
4     while low <= high:
5         mid = (low + high) // 2
6
7         if arr[mid] == target:
8             return mid
9         elif arr[mid] < target:
10            low = mid + 1
11        else:
12            high = mid - 1
13
14    return -1
15 books = [10, 20, 30, 40, 50, 60, 70]
16 k = 40
17 print(binary_search(books, k))
```

Input

stdin input

Output